

HUM24-2

Law Fundamentals & GDPR

Legal systems, contracts, labor, IP, data protection, and EU digital regulation

Law is institutional architecture: designed by humans, encoding values, always incomplete.

Albert School — 12 hours

Preface

You have spent your life inside the law the way a fish lives inside water — so surrounded by it that you rarely notice it is there. You signed terms of service without reading them, accepted cookie banners by reflex, took a job under a contract you skimmed, and streamed music whose copyright you never thought about. It is tempting to conclude that law is a thick book of rules that lawyers memorise, and that your job is to stay out of trouble by deferring to them.

This course asks you to see law differently — as **institutional architecture**. Every legal rule was **designed** by someone, to solve a problem, in pursuit of a goal, after a political and philosophical choice about what matters. Nobody discovered contract law the way we discovered gravity; humans built it, and they built it one way rather than another for reasons you can reconstruct and argue with. Because it was designed by people under constraints, it is also **always incomplete**: it lags behind technology, leaves gaps, and embeds the compromises of its authors.

That single shift — from “law as a list of rules to obey” to “law as designed architecture that encodes values” — is the spine of everything here. We will never start from doctrinal history. We will start from a **concrete business scenario**, ask what problem the law was built to solve, and only then state the rule in the abstract. By the end you should be able to look at a new business activity — launching a data product, hiring across two countries, deploying a hiring algorithm — and reason about which rules apply, which risks are most likely, and what to do about them.

No prior legal knowledge is assumed. What is assumed is the willingness to ask, of every rule, the architect’s question: *who built this, to solve what, and at whose expense?* This course is the entry point to the Law & Regulation thread: Digital Threats & Cybersecurity (S3) and Public Policy & Institutional Analysis (S6) both build on the foundation laid here, and GDPR will recur as the keystone example of regulation as values-encoding design.

Contents

Preface	2
1 Law as Institutional Architecture	4
1.1 A dispute, two courthouses	4
1.2 Two design philosophies	4
2 Contract Law	6
2.1 The promise the state will enforce	6
2.2 Formation: when does a contract come into being?	6
2.3 Validity: is the contract a good one?	7
2.4 Performance, breach, and remedies	7
2.5 The contract types you will meet first	8
3 Labor Law	8
3.1 Why labor law exists at all	8
3.2 The decisive question: employee or contractor?	9
3.3 Protections that constrain hiring and management	9
3.4 A wrongful-dismissal scenario	10
4 Intellectual Property for Technology Companies	11
4.1 Owning things you cannot touch	11
4.2 The risks that actually bite	11
5 GDPR as Institutional Design	13
5.1 A regulation built to encode a value	13
5.2 The core architecture	13

5.3	The six lawful bases	14
6	GDPR in Practice	15
6.1	From principle to product change	15
6.2	Enforcement and the cost of getting it wrong	16
7	EU Digital Regulation: AI Act, DSA, DMA	16
7.1	Three more machines, three more problems	16
7.2	When all the regimes fire at once: a contested AI deployment	17
8	Legal Risk Mapping	18
8.1	Putting the four bodies of law together	18
9	Legal Reasoning as Structured Analysis	19
9.1	The method underneath every chapter	19
9.2	Closing	20

1 Law as Institutional Architecture

1.1 A dispute, two courthouses

A French software company, *Solvio*, signs a contract to supply an analytics platform to an English retailer, *Marwick*. The deal sours: Marwick says the software is too slow and refuses to pay; Solvio says the software meets the agreed specification and sues for the money. The facts are identical in every respect but one — **which country’s law governs the contract**. Litigated in France, the dispute is argued from the general principles of the French Civil Code. Litigated in England, it is argued from a long line of previously decided cases about what “fit for purpose” has meant in other software disputes. Same facts, same emails, same invoice — and yet the way a lawyer prepares, the arguments that count, and the likely outcome all differ.

This is the first lesson of the course in one story: a legal system is not a neutral backdrop. It is a **designed structure**, and the design has consequences that reach all the way down to who pays whom.

1.2 Two design philosophies

The world’s legal systems cluster into two great families. They are not “right” and “wrong”; they are two different answers to the same engineering question: *where should legal certainty come from?*

Definition 1 (Civil law vs common law)

- **Civil law** systems organise the law around comprehensive, enacted **codes**. The primary source of law is the written code; a judge’s task is to find the relevant provision and apply it. Judicial decisions resolve the case at hand but do not, in principle, make binding law for future cases. (France, Germany, Spain, most of continental Europe, Latin America, and — by descent — much of the world.)
- **Common law** systems develop the law incrementally through **judicial decisions**. Past decisions bind later courts on similar facts — the doctrine of **precedent** (*stare decisis*, “to stand by things decided”). Statutes exist and are important, but they sit alongside a vast body of judge-made case law. (England & Wales, the United States, Canada, Australia, India, and other former British jurisdictions.)

The contrast is best seen as a choice about **where you look for the answer**. In a civil law system you reason *from the code down to the facts*: start with the general rule the legislator wrote, then apply it. In a common law system you reason *from past cases across to your facts*: find the closest previously decided case and argue by analogy.

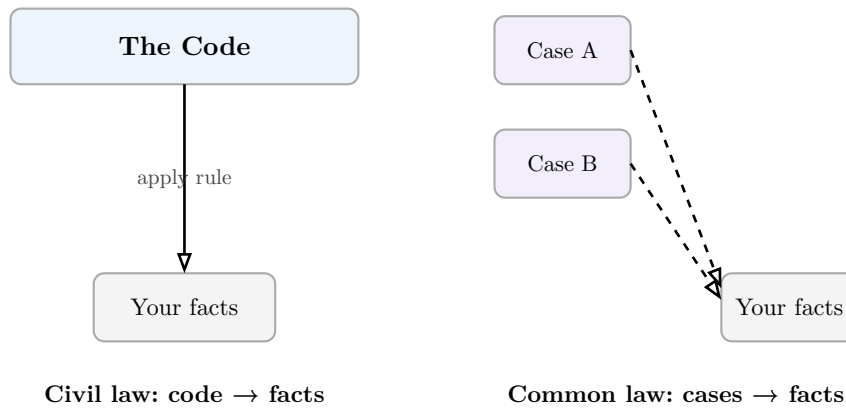


Figure 1: Two routes to an answer. Civil law reasons downward from an enacted code; common law reasons across by analogy from previously decided cases. The same facts enter different machinery.

1.2.1 Business consequences

The design choice is not academic — it changes how firms behave.

- **Contract drafting.** Common law contracts tend to be long and exhaustively detailed, because the parties cannot rely on a code to fill gaps — they must write down everything they want to bind. Civil law contracts can be shorter, because the code supplies default rules in the background.
- **Predictability.** Civil law offers certainty through an accessible written code; common law offers certainty through accumulated precedent, which can be more responsive to novel situations but harder for a non-lawyer to look up.
- **Choice of law.** Because the governing system shapes outcomes, sophisticated parties negotiate two clauses explicitly: a **choice-of-law** clause (which country’s law governs) and a **jurisdiction** clause (which country’s courts hear disputes). In our Solvio–Marwick story, these clauses may matter more than any other line in the contract.

Worked example — Why the choice-of-law clause is worth fighting over. Solvio’s lawyer wants French law and French courts; Marwick’s wants English law and English courts. Neither is being stubborn. Each knows that “fit for purpose” and the remedies for late or defective software have a settled meaning in their own system, and a less predictable one in the other’s. The negotiation over a single boilerplate clause is really a negotiation over which architecture will judge them — and therefore over the likely outcome of any future dispute.

Common pitfall Do not equate civil law with “no role for judges” or common law with “no written statutes.” Civil law judges interpret codes with great sophistication (and pay attention to prior rulings in practice), and common law legislatures pass enormous volumes of statute. The difference is one of **primary source and binding force** — where the law’s authority chiefly resides — not a caricature of judges versus legislators.

Chapter takeaways. A legal system is designed architecture, not a neutral backdrop. Civil law reasons from enacted codes (code → facts); common law reasons from binding precedent (cases → facts). The choice has real business consequences — contract length, predictability, and the often-decisive choice-of-law and jurisdiction clauses.

2 Contract Law

2.1 The promise the state will enforce

You agree to build a friend a website for €2,000. Halfway through, they change their mind and refuse to pay for the work done. Can you make them? The answer turns on whether what you had was a mere arrangement between friends or a **contract** — a promise the state will stand behind with its courts and its power. Contract law is the architecture that decides which promises get that backing, and what happens when one is broken. It is the legal backbone of commerce: almost every business interaction — a sale, a hire, a subscription, a partnership — is a contract.

Definition 2 (Contract) A **contract** is a legally enforceable agreement. “Enforceable” is the operative word: not every promise is a contract, and the law sets out the conditions a promise must meet before a court will lend its force to it.

A contract has a life cycle, and most disputes can be located at one of its stages. We follow that life cycle: **formation** (does a contract exist?), **validity** (is it a good contract?), **performance** (are the parties doing what they promised?), **breach** (has someone failed?), and **remedies** (what does the injured party get?).

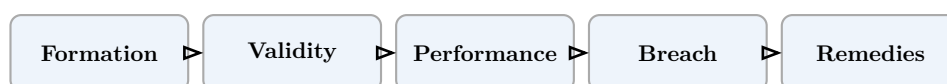


Figure 2: The life cycle of a contract. Most contractual disputes can be placed at exactly one of these stages — locating it is the first step of the analysis.

2.2 Formation: when does a contract come into being?

Return to the website. You said “I’ll build your site for €2,000”; your friend said “Deal.” Money was to change hands. Those three things map onto the three classical elements of formation.

Definition 3 (The elements of formation) A contract is generally formed when there is:

- **Offer** — a definite proposal to be bound on stated terms (“I’ll build your site for €2,000”).
- **Acceptance** — an unqualified agreement to exactly those terms (“Deal”). A reply that changes the terms is not an acceptance but a **counter-offer**, which the other side may then accept or reject.
- **Consideration** (common law) / **cause** (civil law) — something of value moves in each direction. In common law this is **consideration**: each side must give something (you give the work, your friend gives €2,000). In civil law systems the analogous role is played by the **cause** — the reason the obligation exists. This element is what separates a binding bargain from a one-sided gift promise.

Worked example — Offer versus invitation to treat. A price tag on a laptop in a shop window saying “€499” looks like an offer, but in law it is usually an **invitation to treat** — an invitation for **you** to make the offer. You offer €499 at the till; the shop accepts (or, if the tag was a mistake, declines). Why does this technicality matter? Because if the shop window were a binding offer, a pricing error visible to a million online shoppers could form a million contracts in an afternoon. The rule is a piece of deliberate architecture protecting sellers from being bound by an advertisement to the whole world.

Common pitfall The most common student error is to assume that a contract requires a signed document. It does not. Most everyday contracts are oral or formed by conduct (buying coffee, clicking “I agree”). A **signature** is strong evidence that a contract exists and proof of its terms — but the contract is the **agreement**, not the paper. Conversely, calling something a “contract” and signing it does not make it enforceable if an element of formation or validity is missing.

2.3 Validity: is the contract a good one?

A contract can be perfectly formed and still fail, because the law refuses to enforce agreements that are tainted. **Validity** asks whether the agreement deserves enforcement at all.

Definition 4 (Conditions of validity) Even a formed contract is valid only if, broadly:

- the parties have **capacity** (e.g. they are adults of sound mind — a contract with a young child is not enforceable against the child);
- **consent is genuine** — not vitiated by **fraud** (deliberate deception), **error** (a fundamental mistake about what was agreed), or **duress** (consent extracted by illegitimate pressure);
- the **object is lawful** — you cannot make an enforceable contract to do something illegal.

Worked example — Consent that looks present but is not. A supplier signs a renewal only because the customer threatens to leak fabricated allegations unless they do. On paper there is offer, acceptance, and consideration — formation is flawless. But the consent was obtained by **duress**, so the contract is voidable: the law looks past the signature to the conditions under which it was given. Formation asks “did they agree?”; validity asks “was that agreement worth honouring?”

2.4 Performance, breach, and remedies

Definition 5 (Performance, breach, remedy)

- **Performance** is each party carrying out its obligations as agreed.
- **Breach** is a failure to perform an obligation (not performing, performing late, or performing defectively). Not all breaches are equal: a **fundamental** breach goes to the heart of the contract; a minor one does not.
- **Remedies** are what the law gives the injured party. The principal remedy is **damages** — money to put the injured party in the position they would have been in had the contract been performed. In some cases a court may order **specific performance** (actually doing the promised act, used where money is inadequate) or allow **termination** of the contract (releasing the injured party from their own obligations).

Worked example — Choosing the remedy fits the breach. Solvio delivers the analytics platform two weeks late. If the delay cost Marwick €10,000 in lost reporting, the natural remedy is **damages** of €10,000 — money makes Marwick whole. But suppose the contract were for a unique, irreplaceable asset — say a specific painting — and the seller refused to hand it over. Damages cannot buy an identical painting, so a court might order **specific performance**. The remedy is engineered to match the nature of the loss.

Common pitfall The purpose of contract damages is **compensation**, not punishment. A common misconception is that a court will award a large sum to “punish” a party for breaking a contract. In general it will not — damages aim to cover the injured party’s actual loss, no more. Punishment is the job of criminal law, a different architecture with a different goal.

2.5 The contract types you will meet first

In your first roles you will encounter a small set of recurring contract types. Knowing the **job** each one does lets you read it critically rather than signing on reflex.

Definition 6 (Common contract types)

- **Service contract** — one party provides a service for a fee (a consultancy engagement, a maintenance agreement). Key terms: scope, deliverables, fees, liability.
- **Sale contract** — transfer of goods for a price (warranties and risk of loss matter here).
- **Employment contract** — work performed under the **direction** of an employer; this triggers the whole machinery of labor law (Chapter 3).
- **NDA** (non-disclosure agreement) — an obligation to keep specified information confidential. Watch the definition of “confidential information” and the duration.
- **SaaS terms** — the terms of service governing use of a software product, typically covering licence scope, uptime, data handling, liability caps, and termination.

Exercises

1. For each of the following, identify whether a contract was formed, and if not, which element is missing: (a) a billboard reading “Best pizza in town — €8”; (b) “I’ll pay you €500 to paint my fence” / “Sure, this weekend”; (c) a parent promising their child €100 “if you’re good this year.”
2. A company signs a supplier contract after the supplier misrepresented its delivery capacity. Is this a formation problem or a validity problem? Name the vitiating factor and state the likely consequence.
3. You are handed a SaaS contract and an NDA on your first day. List the three terms you would read first in each, and say what business risk each term controls.

Chapter takeaways. A contract is a promise the state will enforce. Its life cycle runs formation → validity → performance → breach → remedies, and most disputes sit at one stage. Formation needs offer, acceptance, and consideration/cause; validity needs capacity, genuine consent, and a lawful object. Remedies (chiefly damages) compensate, not punish. Learn the recurring contract types — service, sale, employment, NDA, SaaS — by the job each does.

3 Labor Law

3.1 Why labor law exists at all

Imagine labor relations governed by **ordinary contract law alone**. An employer and a worker would simply negotiate terms as equals, and whatever they agreed would bind them. The trouble is that they are **not** equals. The worker needs the job to eat; the employer can usually find

another worker. That imbalance — economists call it a **power asymmetry** — means that “free” negotiation would systematically produce terms favouring the stronger party: longer hours, less pay, dismissal at a whim.

Labor law is the institutional response to that asymmetry. It deliberately **overrides freedom of contract** in specific places, imposing protections that the parties **cannot** contract away even if both say they want to. This is its defining architectural feature, and it explains everything that follows: a worker cannot validly “agree” to less than the minimum protections, because the whole point is to remove those terms from the bargaining table.

Definition 7 (Labor law as a corrective) Labor law is the body of rules governing the employment relationship. Unlike ordinary contract law — which assumes roughly equal parties freely bargaining — labor law assumes a **power asymmetry** between employer and worker and imposes **mandatory protections** that override what the parties might otherwise agree.

3.2 The decisive question: employee or contractor?

Almost everything in labor law switches on a single classification.

Definition 8 (Employee versus independent contractor) An **employee** works under the **direction and control** of an employer and is integrated into the organisation; an **independent contractor** runs their own business and supplies a service to a client. Employee status triggers labor-law protections and employer obligations; contractor status does not. Crucially, the classification follows the **substance** of the relationship — who controls how, when, and where the work is done; economic dependence; integration into the firm — **not the label the parties put on it**.

Worked example — The label loses to the reality. A delivery platform calls its riders “independent contractors”: they sign a contractor agreement, invoice the platform, and are told they are running their own micro-business. But the app dictates their routes, sets their pay, monitors their speed, penalises refusals, and forbids subcontracting. A court applying the **substance** test may find that the degree of control is that of an **employer** — and reclassify the riders as employees, regardless of the contract’s wording. The platform then owes everything employee status entails. The lesson: in labor law, calling a duck a chicken does not make it lay eggs.

Common pitfall The single most expensive labor-law mistake a growing company makes is **misclassification** — treating workers as contractors to avoid the cost and obligations of employment, when the reality of control makes them employees. Because classification follows substance, the saving is an illusion: a later reclassification brings back-pay, unpaid social contributions, penalties, and the protections that were bypassed. Never reason from the label you chose; reason from how the work is actually controlled.

3.3 Protections that constrain hiring and management

Once someone is an employee, a set of protections attaches that the employer must work around. These are the constraints a manager actually feels.

Definition 9 (Core employee protections)

- **Termination rules.** An employer generally cannot dismiss **at will**. A lawful dismissal must usually meet a **substantive** requirement (a valid reason — e.g. misconduct, economic redundancy) **and** a **procedural** one (notice, process, sometimes consultation). A dismissal failing either is **wrongful** (or “unfair”) and exposes the employer to compensation or reinstatement.
- **Working time.** Limits on maximum hours, and entitlements to rest periods, breaks, and paid leave.
- **Non-compete clauses.** A clause restricting where an ex-employee may work afterwards is **enforceable only within limits** — it must be reasonable in **scope, duration, and geography**, protect a **legitimate interest**, and in many jurisdictions be **compensated**. Beyond those limits it is void and unenforceable.

Worked example — A non-compete that goes too far. An employer makes a junior analyst sign a clause barring them from working “for any company in any related field, anywhere in the world, for five years” after leaving. This is almost certainly **unenforceable**: it is disproportionate in duration and geography, sweeps far beyond any legitimate interest, and (in many systems) is uncompensated. A court will likely strike it down entirely or read it down. A narrow clause — “not to join a named direct competitor, in this country, for six months, with compensation” — stands a far better chance. The art is restraint: an over-broad clause protects **nothing** because it protects too much.

Employers carry corresponding **obligations**: lawful written contracts, payment of wages and social contributions, a safe working environment, and non-discrimination in hiring and management.

3.4 A wrongful-dismissal scenario

Worked example — Misclassification meets wrongful dismissal. A startup engages a “freelance” designer full-time for eighteen months: fixed hours, a company laptop, daily direction, no other clients. When funding tightens the startup ends the arrangement overnight with no notice. The designer claims they were really an **employee**. Applying the substance test (control, integration, economic dependence), a court finds for the designer. Two consequences follow at once: (1) the startup owes the back-dated employer obligations it skipped; and (2) ending the relationship abruptly was a **wrongful dismissal**, because an employee was owed a reason and a process. One mislabelled relationship triggers two distinct liabilities — a vivid illustration of why the classification question is asked first.

Exercises

4. List five factors a court would weigh to decide whether a worker is an employee or a contractor. For a ride-hailing driver, mark which way each factor points.
5. Rewrite the over-broad non-compete from the worked example into a clause with a realistic chance of being enforced, and justify each limit you impose.
6. An employer dismisses an employee for a genuine reason but with no notice or process. Is the dismissal lawful? Explain using the substantive/procedural distinction.

Chapter takeaways. Labor law exists to correct the **power asymmetry** between employer and worker, imposing protections the parties cannot contract away. The pivotal question is **employee vs contractor**, decided by the substance of control, not the label — and misclassification is the costly trap. Employee status triggers termination rules (substantive + procedural), working-time limits, and non-competes enforceable only within reasonable limits, plus employer obligations.

4 Intellectual Property for Technology Companies

4.1 Owning things you cannot touch

A software company's most valuable assets are often invisible: the code, the algorithm, the brand, the secret method. Intellectual property (IP) law is the architecture that lets a firm **own** these intangibles — granting limited exclusive rights so that creators can capture the value of what they make rather than have it copied freely. The design tension is built in: too little protection and no one invests in creating; too much and society is locked out of ideas. Each IP regime strikes that balance differently.

Definition 10 (The four IP regimes)

- **Patent** — protects a **novel, non-obvious, useful invention** (a technical solution) for a limited term (around 20 years), in exchange for **public disclosure** of how it works. Strong but expensive, slow to grant, and territorial.
- **Trademark** — protects **signs that identify commercial origin** (names, logos, slogans). Can last indefinitely if renewed and used. Protects the **brand**, not the product's function.
- **Copyright** — protects **original works of authorship** — including **software source code** — **automatically**, with no registration required, for a long term. Protects the **expression**, not the underlying idea.
- **Trade secret** — protects **valuable confidential information** (a recipe, an algorithm, a customer list) for as long as it stays secret **and** the holder takes **reasonable measures** to protect it. No registration, no expiry — but no protection at all once the secret is out.

Regime	Protects	Lasts
Patent	a novel invention	~20 yrs, then public
Trademark	a brand sign / logo	indefinite if renewed
Copyright	expression, incl. code	long, automatic
Trade secret	secret information	while kept secret

Figure 3: The four IP regimes at a glance. They protect different things for different durations — and a single product often relies on several at once (patented hardware, copyrighted code, a trademarked name, secret tuning).

4.2 The risks that actually bite

For a technology company, three exposures recur far more often than the rest. Learn these by their business consequence.

Definition 11 (The three high-frequency IP risks)

1. **Open-source licence compliance.** Almost all modern software incorporates open-source components, each distributed under a **licence** with conditions. Using the code while violating those conditions is **copyright infringement**. “Permissive” licences (MIT, Apache) ask little; **copyleft** licences (the GPL family) require that derivative works distributed to others be released under the same open terms. Quietly building a proprietary product on copyleft code can force you to open-source your own code or stop shipping.
2. **Patent exposure.** You may infringe someone’s patent even if you invented your feature independently and never saw theirs — patents bar **use**, not just copying. A particular danger is the **patent troll** (more politely, a non-practising entity): a firm that holds patents not to build products but to extract settlements by threatening litigation, betting that paying up is cheaper than fighting.
3. **Trade secret protection.** Your secret sauce is protected **only** while it is secret and you take **reasonable measures** to keep it so. The risks are **theft** (a departing employee walks out with the source code or customer list) and **self-inflicted loss** (publishing or failing to lock down the secret destroys the protection).

Worked example — A copyleft violation with teeth. A startup builds a slick proprietary networking appliance and, to save time, embeds a chunk of GPL-licensed code in its firmware — without releasing its own source. A copyleft enforcement group notices and sues. The startup faces a stark choice: comply by **releasing its proprietary source code** under the GPL (handing rivals its work), or stop shipping the product until the offending code is removed and re-engineered. Either way the “free” code turned out to be the most expensive in the build. This is the canonical open-source compliance failure.

Common pitfall “Open source is free” is a dangerous half-truth. Open-source code is free of **charge** but not free of **obligation**. Treating an OSS licence as a no-strings gift — rather than a contract whose conditions (attribution, source disclosure, share-alike) must be honoured — is among the most common and consequential IP mistakes a software company makes. Track your dependencies and their licences the way you track your debts.

Worked example — Why a trade secret needs a fence. A company keeps its ranking algorithm as a trade secret but stores it in a company-wide readable wiki, with no access controls and no confidentiality clauses in employee contracts. A former engineer takes it to a competitor. When the company sues, it discovers a deeper problem: because it took **no reasonable measures** to keep the information secret, it may have **no trade secret to protect at all**. The legal protection was conditional on a fence the company never built.

Exercises

7. For each asset, name the IP regime(s) that best protect it and why: (a) the source code of an app; (b) the app’s name and logo; (c) a genuinely new data-compression method; (d) the tuned parameters that make a model perform well.
8. A team wants to ship a product that depends on a GPL-licensed library. State the compliance question they must answer before launch, and the two outcomes if they get it wrong.

9. List four concrete “reasonable measures” a company should take so that its confidential information actually qualifies as a protectable trade secret.

Chapter takeaways. IP law lets firms own intangibles via four regimes — patent, trademark, copyright (automatic, covers code), and trade secret (conditional on secrecy). For tech companies the three biting risks are open-source licence compliance (a copyleft violation can force you to open-source or stop shipping), patent exposure (including trolls — independent invention is no defence), and trade-secret protection (no fence, no secret).

5 GDPR as Institutional Design

5.1 A regulation built to encode a value

Most people meet the GDPR as an annoyance: the cookie banner, the consent pop-up, the privacy policy nobody reads. Seen that way, it is bureaucracy. Seen correctly, it is the clearest example in this course of **regulation as values-encoding institutional design** — which is why it is our keystone case.

The European Union made a **choice**: that control over one’s personal data is an aspect of human **privacy and dignity** that a person should retain even after handing data to a company. That is a philosophical and political commitment, not a technical necessity — other jurisdictions chose differently, treating data more as a freely tradable commodity. The General Data Protection Regulation (GDPR) is the machine that turns that abstract European commitment into **enforceable obligations** on anyone who processes personal data. To understand any specific rule, ask what value it is encoding.

Definition 12 (GDPR, and why it exists) The **General Data Protection Regulation** (GDPR) is the EU’s regulation governing the **processing of personal data** — any information relating to an identifiable person. It exists as a deliberate institutional design encoding a European commitment: that individuals retain rights over their personal data, and that those who process it bear enforceable duties. It treats data protection as a matter of fundamental rights, not merely consumer protection.

A few terms recur and are worth fixing now: the **data subject** is the individual the data is about; the **controller** is the entity that decides why and how data is processed (and bears the primary obligations); the **processor** acts on the controller’s instructions.

5.2 The core architecture

Four load-bearing principles hold the whole structure up.

Definition 13 (The four pillars of the GDPR)

- **Lawful basis.** Processing personal data is lawful **only if** it rests on one of six enumerated bases — you must pick the right one **before** processing (next section).
- **Data subject rights.** Individuals hold enforceable rights over their data: **access**, **rectification** (correction), **erasure** (“right to be forgotten”), **restriction**, **portability** (take your data elsewhere), and **objection**.

- **Data minimisation.** You may collect and keep only data that is **adequate, relevant, and limited to what is necessary** for a specified purpose — and keep it no longer than needed. Collecting “just in case” is a violation.
- **Accountability.** It is not enough to comply; the controller must be able to **demonstrate** compliance (documentation, records, impact assessments). The burden of proof sits with the processor of data.

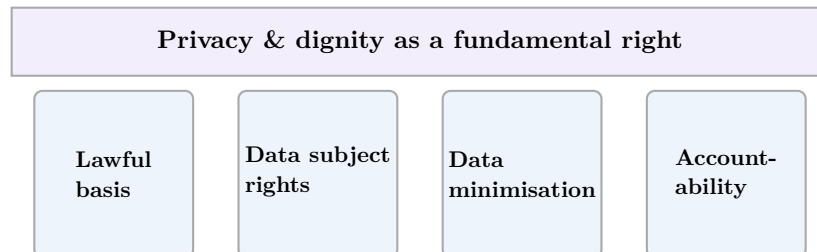


Figure 4: The GDPR as architecture: four pillars (lawful basis, data subject rights, minimisation, accountability) hold up a single value — that privacy and dignity are fundamental rights, not commodities.

5.3 The six lawful bases

The first question a firm must answer about any processing is: *on what basis?* There are exactly six, and the art is choosing the right one — and only one — per purpose.

Definition 14 (The six lawful bases (Article 6))

1. **Consent** — the data subject has freely given specific, informed, unambiguous agreement.
2. **Contract** — processing is necessary to perform a contract with the data subject (e.g. shipping an address to deliver an order).
3. **Legal obligation** — processing is required by law (e.g. retaining invoices for tax).
4. **Vital interests** — necessary to protect someone’s life.
5. **Public task** — necessary for an official function in the public interest.
6. **Legitimate interests** — necessary for the controller’s (or a third party’s) legitimate interests, **except** where overridden by the data subject’s rights — a balancing test.

Worked example — Picking the basis — and why consent is often the wrong one.

An online shop processes a customer’s address. The intuitive answer is “get consent.” But to **deliver the order**, the correct basis is **contract** — processing the address is necessary to perform the sale, so no separate consent is needed (and asking for it would be misleading, since the customer cannot refuse and still get their parcel). If the same shop wants to **email marketing** to that customer, **that** purpose typically needs **consent**, because it is not necessary to the sale. One customer, two purposes, two different lawful bases — the analysis is **per purpose**, not per company.

Common pitfall Consent is **not** the default or “safest” lawful basis — it is often the hardest to rely on, because it must be freely given (no pre-ticked boxes, no “consent or no service” for non-essential processing), specific, informed, and **as easy to withdraw as to give**. Reaching for consent when **contract** or **legitimate interests** fits better creates a

fragile basis the user can withdraw at any moment. Choose the basis that actually fits the purpose.

Chapter takeaways. GDPR is the keystone example of regulation encoding a value — that privacy and dignity are fundamental rights. Its architecture rests on four pillars: lawful basis, data subject rights, data minimisation, and accountability. Every processing activity needs one of six lawful bases, chosen **per purpose**; consent is one option among six and frequently not the best fit.

6 GDPR in Practice

6.1 From principle to product change

Principles only matter when they reshape what a team actually builds. Suppose you are launching a fitness app that collects names, emails, location traces, and health metrics. GDPR is not a document your lawyer files; it is a set of **concrete changes** to how the product collects, stores, processes, and obtains consent.

Definition 15 (GDPR across the data lifecycle) For a specific digital product, compliance requires concrete changes at each stage:

- **Collection** — collect only what is necessary for a defined purpose (minimisation), and identify a **valid lawful basis for each purpose** before collecting. No “just in case” fields.
- **Storage** — keep data only as long as necessary (retention limits), store it securely (encryption, access control), and know **where** it lives — including transfers outside the EU, which carry extra constraints.
- **Processing** — use data only for the purposes for which the basis was established (**purpose limitation**); repurposing data for something new may require a fresh basis.
- **Consent architecture** — where consent is the basis, it must be freely given, specific, informed, and **as easy to withdraw as to give**. This dictates real UI: granular toggles (not one “accept all”), no pre-ticked boxes, an accessible withdrawal mechanism, and a record of what each user consented to (accountability).

Worked example — What changes in the fitness app. Before GDPR thinking, the app collects everything by default and buries one “I agree” checkbox. Compliant design forces specific changes: the **health metrics** (a special, sensitive category) need an explicit, separate basis and heightened protection; the **location trace** is collected only when a workout is active, not continuously (minimisation); marketing emails get a **separate, unticked** opt-in (consent), while order/account data rides on **contract**; a settings screen lets the user **withdraw** consent and **export or delete** their data (data subject rights). Each pillar from Chapter 5 turns into a UI decision or a stored record.

Common pitfall A pre-ticked consent box, a single “Accept all” with no real “Reject” of equal prominence, or “agree to everything or you cannot use the app” (for non-essential processing) are **not valid consent**. Designing the consent flow as a dark pattern to maximise opt-ins is a frequent and now heavily-fined mistake — enforcement authorities treat the **design** of the consent interface as part of whether consent was truly free.

6.2 Enforcement and the cost of getting it wrong

A rule without teeth changes nothing. The GDPR’s teeth are **supervisory authorities** (one per member state) with power to investigate, order changes, and impose **administrative fines scaled to global turnover**.

Definition 16 (The enforcement landscape) GDPR is enforced by national **supervisory authorities** (data protection authorities). They can investigate complaints, audit controllers, order processing to change or stop, and levy **administrative fines** — up to the higher of a large fixed cap or a **percentage of the firm’s worldwide annual turnover** for the most serious breaches. The consequences of non-compliance are therefore **financial and operational**, not merely reputational.

Worked example — Why the turnover-based fine is itself a design choice. Scaling the maximum fine to **global turnover** rather than to a fixed sum is deliberate architecture. A flat €1M cap would be a rounding error to a trillion-euro platform — a licence fee for ignoring the law. By tying the ceiling to turnover, the regulation makes non-compliance **proportionately** painful for the largest firms, encoding the value that the rules bind the powerful as much as the small. The penalty structure is itself an act of institutional design.

Exercises

10. For a food-delivery app, list every category of personal data it plausibly processes, assign each a lawful basis, and flag any data you would **not** collect under minimisation.
11. Critique a consent banner you have seen recently. Identify at least two ways in which it does or does not meet the “freely given, specific, informed, withdrawable” standard.
12. Explain, to a product manager, why scaling fines to turnover changes the incentive of a very large company more than a fixed cap would.

Chapter takeaways. GDPR compliance is concrete product change across the data lifecycle: minimised, lawfully-based **collection**; secure, time-limited **storage**; purpose-limited **processing**; and a genuine **consent architecture** (granular, unticked, withdrawable, recorded). Enforcement by supervisory authorities, with fines scaled to global turnover, makes non-compliance a financial and operational risk — and the turnover-based penalty is itself a values-encoding design choice.

7 EU Digital Regulation: AI Act, DSA, DMA

7.1 Three more machines, three more problems

GDPR was the first major piece of the EU’s digital rulebook; it is not the last. Three further regulations extend the same logic — each **built to address a specific business problem** and each **encoding a specific value into an enforceable rule**. We preview them here (they are developed in HUM34-3 and HUM64-2). The trick to remembering them is to learn each by the **problem it solves**, not its acronym.

Definition 17 (The AI Act, DSA, and DMA)

- **AI Act.** Problem: AI systems can cause harm at scale and opaquely. Design: regulate AI by **risk tier** — **prohibited** uses (e.g. social scoring), **high-risk** systems (subject to strict requirements: risk management, data governance, human oversight, transparency), and **limited/minimal-risk** systems. Imposes obligations on **providers** and **deployers** of high-risk systems. Value encoded: higher-risk uses must carry proportionate safeguards.
- **DSA** (Digital Services Act). Problem: illegal and harmful content spreads on online platforms with no accountability. Design: graduated obligations on platforms for **content moderation**, notice-and-action on illegal content, transparency, and (for the very largest) systemic-risk audits. Value encoded: platforms are **accountable** for the systems they run.
- **DMA** (Digital Markets Act). Problem: a few “gatekeeper” platforms entrench market power and foreclose competition. Design: **ex-ante** obligations on designated **gatekeepers** (no self-preferencing, interoperability, data portability). Value encoded: digital markets must stay **contestable and fair**.

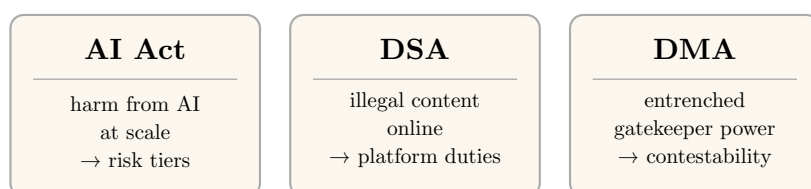


Figure 5: Learn each regulation by the problem it solves. AI Act → risk of AI harm; DSA → unaccountable platforms; DMA → entrenched market power. Each turns a value choice into an enforceable obligation.

7.2 When all the regimes fire at once: a contested AI deployment

The regulations are not silos. A single contested deployment can trigger several at the same time — and integrating them is exactly the skill this thread builds.

Worked example — An algorithmic hiring tool, analysed across regimes. A company deploys an AI system that screens job applicants and auto-rejects those below a score. Three legal layers apply simultaneously:

1. **GDPR lawful basis.** The system processes applicants’ personal data; the controller must identify a valid basis. **Consent** is weak here (an applicant cannot freely refuse and still be considered), so the basis is more likely **legitimate interests** or **contract-preparatory processing** — and it must be documented.
2. **AI Act tier and deployer obligations.** AI used for recruitment and employment decisions is classified **high-risk**. The deployer therefore owes obligations such as human oversight, monitoring, transparency to candidates, and use within the provider’s instructions.
3. **Article 22 GDPR.** This article restricts decisions based **solely on automated processing** that produce legal or similarly significant effects on a person — and being auto-rejected from a job qualifies. The applicant is entitled to **human intervention**, an explanation, and the ability to contest. A purely automatic reject with no human in the loop is the precise thing Article 22 constrains.

The correct analysis names **all three** layers and how they interlock — not just one.

Common pitfall A frequent error is to treat a contested AI deployment as “a GDPR problem” **or** “an AI Act problem” — as if one regime displaces the others. They **stack**. A hiring algorithm can be simultaneously high-risk under the AI Act, processing personal data under the GDPR, and making an Article 22 solely-automated decision. Analysing only one layer leaves the firm exposed on the others.

Exercises

13. For each regulation (AI Act, DSA, DMA), state in one sentence the business problem it addresses and the value it encodes.

14. A bank deploys an automated credit-scoring model that declines applicants with no human review. Identify the GDPR lawful-basis question, the AI Act risk tier, and whether Article 22 applies. Justify each.

15. Why might the same AI content-moderation system implicate both the DSA and the GDPR? Name the distinct concern each regime brings.

Chapter takeaways. Beyond GDPR, three regulations extend the EU’s digital architecture: the **AI Act** (risk tiers + deployer obligations, for AI harm), the **DSA** (platform accountability for content), and the **DMA** (gatekeeper obligations for contestable markets). Learn each by its problem and value. A contested AI deployment **stacks** regimes — GDPR lawful basis, AI Act tier, and Article 22’s limit on solely-automated significant decisions all apply at once.

8 Legal Risk Mapping

8.1 Putting the four bodies of law together

Each previous chapter examined one body of law in isolation. A real business activity ignores those boundaries: launching a data-driven product simultaneously involves contracts (with customers and suppliers), labor law (the team building it), data law (the personal data it uses), and IP (the code and brand it relies on). **Legal risk mapping** is the discipline of looking at one named activity and **integrating** all four to see the whole risk surface — then deciding what to do about it.

Definition 18 (Legal risk mapping) **Legal risk mapping** is a structured analysis of a **named business activity** that:

1. **Identifies the applicable regulations** across all four areas — contract, labor, data, and IP law;
2. **Assesses the highest-probability risks** — not every conceivable risk, but the ones most likely to materialise and most damaging if they do;
3. **Proposes proportionate mitigations** — measures matched to the size and likelihood of each risk (not gold-plating everything, not ignoring the big ones);
4. **Justifies the analysis** using structured legal reasoning (next chapter).

The key word is **proportionate**. A risk map is not a counsel of perfection that demands the firm eliminate every risk — that would paralyse the business. It ranks risks by **probability** × **impact** and spends effort where it pays.

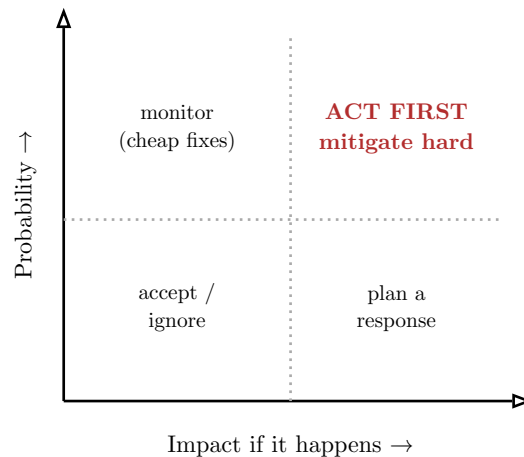


Figure 6: Proportionate mitigation ranks risks by probability $\times$ impact. Spend scarce legal and engineering effort on the upper-right; merely monitor or accept the rest.

Worked example — A risk map for launching a data-driven product. Activity: a startup launches a recommendation product using customer behaviour data, built partly on open-source components, staffed by a mix of employees and freelancers.

- **Contract:** customer terms and SaaS liability caps; supplier data-processing agreements. *Risk:* unlimited liability hidden in a supplier contract. *Mitigation:* negotiate a liability cap.
- **Labor:** the “freelancers” who work full-time under direction. *High-probability risk:* misclassification. *Mitigation:* reclassify or genuinely loosen control.
- **Data:** personal behavioural data drives recommendations. *High risk:* no valid lawful basis; possibly Article 22 if recommendations become significant automated decisions. *Mitigation:* document basis, add consent/objection, human review where decisions are significant.
- **IP:** a copyleft component in the codebase. *High risk:* forced source disclosure. *Mitigation:* audit dependencies, replace or isolate the component.

The map’s value is that it surfaces **all four** at once and ranks them, so the founder fixes the copyleft and misclassification risks **before** launch rather than discovering them in a lawsuit.

Common pitfall Two opposite failures haunt risk mapping. The first is **tunnel vision** — analysing only the lawyer’s favourite area (usually data) and missing the labor or IP landmine. The second is **boiling the ocean** — listing 200 theoretical risks with no ranking, which is as useless as listing none. A good map is **complete across the four areas** but **ruthlessly prioritised** within them.

Chapter takeaways. Legal risk mapping integrates contract, labor, data, and IP law for one named activity: identify applicable rules across all four, assess the **highest-probability** risks, propose **proportionate** mitigations (ranked by probability $\times$ impact), and justify with structured reasoning. Avoid both tunnel vision and boiling the ocean.

9 Legal Reasoning as Structured Analysis

9.1 The method underneath every chapter

You may have noticed that every analysis in this course followed the same hidden shape. We never asked “is this legal?” in the abstract. We asked: *what exactly is in dispute? which rule*

governs it? how does that rule apply to these facts? That three-step shape is **legal reasoning**, and naming it explicitly is the final tool the course gives you — because it is the method you will **reactivate on every new problem** for the rest of the thread.

Definition 19 (The three steps of legal reasoning) Faced with any legal question, proceed in order:

1. **Identify the legal question** — state precisely what is in dispute (not “is this okay?” but “was the worker an employee or a contractor?”).
2. **Locate the applicable norm** — find the rule that governs it: a code provision, a binding precedent, or a regulation (e.g. Article 6 GDPR, the substance test for employment).
3. **Evaluate the likely outcome** — apply the norm to the specific facts and reason to a probable result, acknowledging uncertainty where it exists.

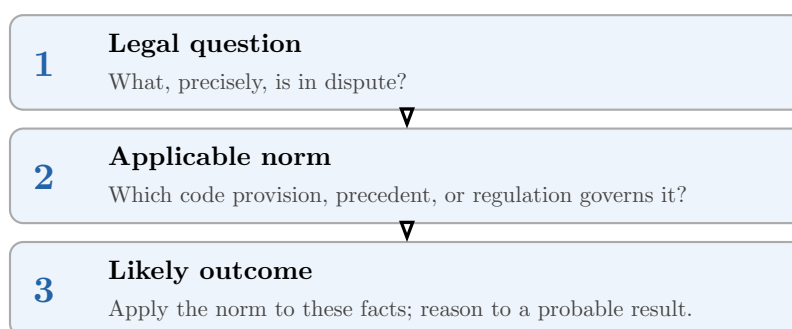


Figure 7: The reasoning engine of the whole course: legal question → applicable norm → likely outcome. Every case analysis and every risk assessment is an instance of this sequence.

Worked example — The engine running on our delivery-rider case.

1. **Legal question:** are the platform’s riders employees or independent contractors?
2. **Applicable norm:** the substance test — classification follows actual control, integration, and economic dependence, not the contract’s label.
3. **Likely outcome:** given that the app dictates routes, sets pay, monitors performance, and forbids subcontracting, the degree of control points to **employment**; a court is likely to reclassify, triggering back-pay and the bypassed protections.

Notice the discipline: a **sharp** question, a **named** norm, and an outcome reasoned **from the facts** — with the residual uncertainty (“likely”) stated honestly.

Common pitfall The amateur’s instinct is to leap straight to the conclusion (“they should win”) and reason backwards. The structure exists to stop that: by forcing you to pin the **question** and name the **norm** before you reach an outcome, it exposes whether your conclusion actually follows from a rule or is just a hunch. An answer that cannot name the norm it rests on is an opinion, not a legal analysis.

9.2 Closing

You arrived seeing law as a wall of rules to obey. You leave, I hope, seeing it as **architecture you can read**: every rule designed by someone, to solve a problem, encoding a value, and never quite complete. Contract law decides which promises the state will back; labor law corrects a power imbalance; IP law lets firms own the intangible; the GDPR turns a commitment to privacy and dignity into enforceable duties; and the AI Act, DSA, and DMA extend that same logic to

new problems. Tying it all together is one method — legal question → applicable norm → likely outcome — and one habit: always ask who built this rule, to solve what, and at whose expense. That habit, not any single rule you have memorised, is what you carry into the rest of the Law & Regulation thread.

Exercises

16. Take any worked example from an earlier chapter and rewrite the analysis explicitly in the three steps: legal question, applicable norm, likely outcome.
17. Pick a recent news story involving a company and a legal dispute. State the legal question in one sentence, name the body of law that governs it, and give your reasoned guess at the outcome — flagging what you would need to know to be more certain.
18. Explain in your own words why “identify the norm” must come **before** “state the outcome,” using an example where skipping the step would mislead you.

Chapter takeaways. Legal reasoning is the engine beneath every chapter: **legal question → applicable norm → likely outcome**. State a sharp question, name the governing rule (code, precedent, or regulation), then reason to a probable result from the facts — stating uncertainty honestly. An analysis that cannot name its norm is opinion, not law. This method, reactivated on every problem, is the course’s lasting tool.